Guía de Modelación Molecular en GROMACS

https://gemini.google.com/app/d0a9dc1af398dc30

User prompt: dame los pasos que tengo que seguir para la modelacion molecular con gromarcs

Response: Un flujo de trabajo estándar en **GROMACS** para la simulación de dinámica molecular (MD) de un sistema biológico (por ejemplo, una proteína en solución acuosa) se compone de seis etapas principales:

1. **Preparación del sistema y topología**

   PDB clean, gmx pdb2gmx

   Obtén el archivo `.pdb` desde el Protein Data Bank y limpia heteroátomos no deseados (como moléculas de agua de cristalización o ligandos si solo simularás la proteína). Genera la topología, los parámetros del campo de fuerza y las coordenadas tratadas:

   ```
   gmx pdb2gmx -f proteina.pdb -o proteina_processed.gro -p topol.top -i posre.itp
   ```

   (Selecciona el campo de fuerza y el modelo de agua adecuados cuando el programa lo solicite, p. ej., OPLS-AA y TIP3P).

2. **Definición de la caja de simulación y solvatación**

   gmx editconf, gmx solvate

   Establece las dimensiones del dominio de simulación manteniendo una distancia mínima al borde (típicamente 1.0 nm) para evitar la auto-interacción periódica, y llena la caja con moléculas de agua:

   ```
   # Definir la caja (cúbica o dodecaedro rómbico)
   gmx editconf -f proteina_processed.gro -o proteina_box.gro -c -d 1.0 -bt cubic

   # Solvatar la proteína
   gmx solvate -cp proteina_box.gro -cs spc216.gro -o proteina_solv.gro -p topol.top
   ```

3. **Adición de iones y neutralización**

   gmx grompp, gmx genion

   Procesa el sistema para generar un archivo de entrada binario `.tpr` y reemplaza moléculas de agua aleatorias con iones (Na^+ / Cl^-) para neutralizar la carga neta del sistema y fijar la fuerza iónica (p. ej., 0.15 M):

   ```
   # Compilar la estructura y la topología con parámetros predeterminados
   gmx grompp -f ions.mdp -c proteina_solv.gro -p topol.top -o ions.tpr

   # Insertar iones sustituyendo solvente (selecciona el grupo SOL)
   gmx genion -s ions.tpr -o proteina_solv_ions.gro -p topol.top -pname NA -nname CL -neutral -conc 0.15
   ```

4. **Minimización de energía (EM)**

   gmx grompp, gmx mdrun

   Relaja la estructura eliminando emparejamientos estéricos (solapamientos atómicos) antes de iniciar la simulación dinámica para evitar inestabilidades numéricas:

   ```
   gmx grompp -f minim.mdp -c proteina_solv_ions.gro -p topol.top -o em.tpr
   gmx mdrun -v -deffnm em
   ```

5. **Equilibrado del sistema (NVT y NPT)**

   Termostato y barostato con restricciones de posición

   Equilibra la temperatura y la presión del solvente manteniendo la proteína restringida mediante dos fases secuenciales:

   - **Fase NVT (Número de partículas, Volumen, Temperatura):** Estabiliza la temperatura objetivo (p. ej., 300 K).

   ```
   gmx grompp -f nvt.mdp -c em.gro -r em.gro -p topol.top -o nvt.tpr
   gmx mdrun -v -deffnm nvt
   ```

   - **Fase NPT (Número de partículas, Presión, Temperatura):** Estabiliza la densidad y presión del sistema (p. ej., 1 bar).

   ```
   gmx grompp -f npt.mdp -c nvt.gro -r nvt.gro -t nvt.cpt -p topol.top -o npt.tpr
   gmx mdrun -v -deffnm npt
   ```

6. **Simulación de producción y trayectoria**

   gmx mdrun en producción

   Elimina las restricciones de posición de la proteína y ejecuta la trayectoria de dinámica molecular libre por el tiempo requerido (nanosegundos o microsegundos):

```
gmx grompp -f md.mdp -c npt.gro -t npt.cpt -p topol.top -o md_0_1.tpr
gmx mdrun -v -deffnm md_0_1
```

Post-procesamiento y análisis

Una vez completada la producción, procesa la trayectoria para corregir las condiciones periódicas de contorno (PBC) y analiza las propiedades estructurales básicas:

- **Corrección de PBC:** `gmx trjconv -s md_0_1.tpr -f md_0_1.xtc -o md_noPBC.xtc -pbc mol -center`
- **Estabilidad estructural (RMSD):** `gmx rms -s md_0_1.tpr -f md_noPBC.xtc -o rmsd.xvg`
- **Flexibilidad de residuos (RMSF):** `gmx rmssf -s md_0_1.tpr -f md_noPBC.xtc -o rmsf.xvg`
- **Radio de giro (R_g):** `gmx gyrate -s md_0_1.tpr -f md_noPBC.xtc -o gyrate.xvg`

User prompt: Muestra comandos optimizados de gmx mdrun para aprovechar la aceleración por GPU CUDA en Linux.

Response: Para maximizar el rendimiento de **`gmx mdrun`** en sistemas Linux equipados con GPUs NVIDIA CUDA, el objetivo principal es descargar (*offload*) las tareas computacionalmente intensivas (interacciones de corto alcance, PME y actualización de integradores) a la GPU, minimizando los cuellos de botella de transferencia de memoria PCIe y los tiempos de espera en la CPU.

Configuración del entorno antes de ejecutar

Antes de llamar a `gmx mdrun`, asegura las variables de entorno para optimizar la asignación de memoria y el enlace de hilos CUDA/OpenMP:

```
# Habilitar CUDA PME/Nonbonded offloading directo si utilizas múltiples GPUs
export GMX_ENABLE_DIRECT_GPU_COMM=1

# Fijar el número de hilos OpenMP por hilo MPI para evitar sobre-suscripción
export OMP_NUM_THREADS=4
```

Comandos `gmx mdrun` según el escenario de hardware

1. GPU única + CPU multi-núcleo (Configuración de escritorio / Workstation)

Descarga las interacciones de corto alcance (Nonbonded), el cálculo PME de largo alcance, los enlaces restringidos (LINCS) y el integrador/actualización de coordenadas a la GPU.

```
gmx mdrun -v -deffnm md_0_1 \
  -nb gpu \
  -pme gpu \
  -bonded gpu \
  -update gpu \
  -pin on \
  -ntomp 4
```

- **`-nb gpu`**: Procesa las interacciones de van der Waals y electrostáticas directas en la GPU.
- **`-pme gpu`**: Ejecuta la transformada rápida de Fourier (FFT) y la malla PME en la GPU.
- **`-bonded gpu`**: Mueve los potenciales enlazados y las restricciones LINCS a la GPU.
- **`-update gpu`**: Mantiene las posiciones y velocidades dentro de la VRAM de la GPU, eliminando la necesidad de transferir coordenadas a la CPU en cada paso temporal.
- **`-pin on`**: Mantiene los hilos OpenMP anclados a núcleos físicos específicos de la CPU para reducir las latencias de caché.

2. GPU única con descomposición PME/PP dedicada (CPU muy potente)

Si dispones de una CPU con un recuento elevado de núcleos, puede ser más eficiente dedicar la GPU exclusivamente a las fuerzas de campo cercano (Particle-Particle, PP) y procesar la malla PME en hilos de CPU dedicados:

```
gmx mdrun -v -deffnm md_0_1 \
  -nb gpu \
  -pme cpu \
  -npme 1 \
  -ntomp 8 \
  -pin on
```

3. Servidor o Nodo Multi-GPU (p. ej., 2x o 4x GPUs CUDA)

Aprovecha el paralelismo MPI / asignación de tareas a múltiples GPUs dentro del mismo nodo, mapeando explícitamente los IDs de los dispositivos CUDA.

```
# Ejemplo para un nodo con 2 GPUs utilizando 2 rangos MPI y 6 hilos OpenMP por rango
mpirun -np 2 gmx_mpi mdrun -v -deffnm md_0_1 \
  -gpu_id 01 \
  -nb gpu \
  -pme gpu \
  -bonded gpu \
  -update gpu \
  -npme 1 \
  -pin on \
  -ntomp 6
```

- **`-gpu_id 01`**: Asigna la GPU 0 al rango MPI 0 y la GPU 1 al rango MPI 1.
- **`-npme 1`**: Asigna un rango MPI exclusivamente para la tarea PME en una de las GPUs, mientras los demás manejan las interacciones PP.

Diagnóstico de rendimiento y ajuste dinámico

Para verificar que el *offload* a CUDA esté operando sin cuellos de botella:

1. **Revisa el archivo de salida `.log`:** Al final de la corrida, revisa las secciones **`R3DFOX / Compute Load Balance`** y **`Cycle and Time Accounting`**. Si el tiempo consumido en *Wait GPU* es elevado, redistribuye el tamaño de la malla PME o ajusta `-ntomp`.
2. **Uso de autotuning dinámico (`-pmefft`):** Permite que GROMACS ajuste automáticamente la carga de trabajo entre el espacio real y el espacio recíproco durante los primeros pasos de la simulación:

   ```
   gmx mdrun -v -deffnm md_0_1 -nb gpu -pme gpu -tune_pme yes
   ```
